pp reference manual

a content-addressed, capability-scoped Lisp

Contents

1 Introduction

pp is a content-addressed, capability-scoped Lisp. It is an experiment in using one evaluation model for work usually split among build systems, supervisors, and orchestrators. Each manages computations, dependencies, reuse, and effects. pp supplies that substrate; toolchains and domain policy remain ordinary libraries and narrow trusted providers.

Two ideas carry the whole design. First, every value has a content hash. So two computations with the same code and inputs are the same computation. Second, side effects require capabilities: unforgeable tokens, granted only at the command line, that code must hold to touch the world. Everything else in this manual follows from those two ideas.

The fastest way to learn pp is to read it running. Every example here is a real program run by the manual build. The output shown is what it actually printed. Here is the smallest example: values, a conditional, and arithmetic.

hello.pp

```
# Every value has a content hash; arithmetic and comparison are variadic.
print(+(1, 2, 3))
print(if 5 > 0 { "positive" } else { "negative" })
print(6 * 7)
```

```
$ pp hello.pp
```

```
6
"positive"
42
```

The output appears below the command because the manual ran that file. If an example ever stopped producing the output shown, the build for this page would have failed.

That extends to mistakes. pp checks optional type annotations when a function's body runs. Feed one the wrong type and it says so, naming the offending value:

`type-error.pp`

```
# Type annotations are checked when the body runs. Passing a string where
# an int is expected is caught — the manual shows the real error.
def increment(n: int): int { n + 1 }
increment("oops")
```

```
$ pp type-error.pp
```

```
pp: error: type mismatch: expected int, got "oops" at type-error.pp:3
```

You will see errors this way throughout the manual: provoked and shown, not described. So you know exactly what pp does when something is wrong, not just when everything is right.

1.1 How to read this manual

The chapters move from the surface inward. Values, bindings, and functions come first, then laziness, then the effect-and-capability system that governs side effects. From there the manual turns to what makes pp a build substrate: content-addressed nodes and the store, foreign execution, modules and islands, domains and the reconciler, and finally distribution. We introduce each construct, give a sentence or two of motivation, then show it by running it.

2 The language in brief

This chapter is enough pp to read the rest of the manual. pp is a Lisp-1: functions and variables share one namespace, and evaluation is inner-first and strict. The language reference appendix has the full detail of every form. Here is the working subset.

2.1 Values

pp has the atoms you expect: integers and floats, strings, the booleans `true` and `false`, `nil`, keywords like `:host`, and quoted symbols like `quote { name }`. Arithmetic and comparison are variadic. `if` is an expression: it returns a value.

`hello.pp`

```
# Every value has a content hash; arithmetic and comparison are variadic.
print(+(1, 2, 3))
print(if 5 > 0 { "positive" } else { "negative" })
print(6 * 7)
```

```
$ pp hello.pp
```

```
6
"positive"
42
```

2.2 Bindings

`let` introduces local bindings. They are mutual: every binding can see every other, whatever order you write them in. For the ordinary top-down behaviour, where each binding sees only the ones above it, use `let*`.

lang-bindings.pp

```
# Bindings in a `let` are mutual: every binding sees every other regardless
# of the order they are written. One flat `let` with multiple bindings is
# the idiomatic style — no nested single-binding ladders.
print(let (y = x + 1, x = 1) { y })

# `let*` is the sequential form, where each binding sees only the ones above.
print(let* (x = 1, x = x + 1) { x })
```

```
$ pp lang-bindings.pp
```

```
2
2
```

Both forms bind names to the values of their right-hand sides. `def` does the same at the top level, and also defines functions.

2.3 Functions

A function definition and a function value are the same idea written two ways. `def` names one; `fn` produces one anonymously. There is no loop keyword: recursion is the loop. The tree-walker eliminates tail calls, so recursion does not grow the stack.

lang-functions.pp

```
# `def` names a function; the last expression is its result.
# Functions are values — `fn` makes one without naming it.
def square(x) { x * x }
print(square(7))

# Inline function as a value.
print((fn(x) { x * x })(7))

# Recursion is the only loop. Flat `else if` chains keep branching clean.
def factorial(n) {
  if n = 0 {
    1
  } else {
    n * factorial(n - 1)
  }
}

print(factorial(5))
```

```
$ pp lang-functions.pp
```

```
49
49
120
```

2.4 Types

Type annotations are optional. When present, pp checks them at the moment a function's body runs, not before. A mismatch names the offending value and location:

`type-error.pp`

```
# Type annotations are checked when the body runs. Passing a string where
# an int is expected is caught — the manual shows the real error.
def increment(n: int): int { n + 1 }
increment("oops")
```

```
$ pp type-error.pp
```

```
pp: error: type mismatch: expected int, got "oops" at type-error.pp:3
```

2.5 Laziness

`delay` builds a thunk: a computation that has not run yet. `force` runs it and memoizes the result. The work happens at most once, however many times you force it.

`lang-laziness.pp`

```
# `delay` builds a thunk that has not run yet; `force` runs it, and the
# result is memoized — the work happens at most once.
let (t = delay(do {
  print("working")
  100 * 200
})) {
  print(force(t))
  print(force(t))
}
```

```
$ pp lang-laziness.pp
```

```
"working"
20000
20000
```

This is more than a convenience. Two identical thunks with the same inputs and environment are the same thunk: computed once, shared everywhere. Wrapping a computation in `node` (the next chapter) extends that sharing across separate runs of the program.

2.6 Effects and handlers

Side effects go through `perform`, which dispatches to the nearest enclosing handler. Handling an effect yourself needs no capability. The authority model in the chapter on capabilities governs only the effects that reach out and touch the world.

lang-effects.pp

```
# A side effect is a `perform` that dispatches to the nearest handler.
# This needs no capability: the authority model governs effects that touch
# the world, not ones you handle yourself. The `handlers:` clause takes a
# map-valued block — `{ :name -> fn, ... }`.
with { handlers: { :ask -> fn(q) { 42 } } } {
  print(perform ask("the answer?"))
}
```

```
$ pp lang-effects.pp
```

```
42
```

2.7 Config

Config is ambient data: dynamically scoped values that any code in the dynamic extent can read. It is deliberately not a capability. Config is information, capabilities are authority, and the manual keeps them apart.

lang-config.pp

```
# Config is ambient, dynamically-scoped data — distinct from capabilities,
# which are authority. A binding is visible to everything called within.
# $config(key) is the idiomatic observation sigil for config reads.
print(with-config({:host -> "db1"}) { $config(:host) })
```

```
$ pp lang-config.pp
```

```
"db1"
```

That is the surface. Everything from here on is about what happens when you wrap a computation in `node` and give pp the authority to act on the world.

3 Nodes and content-addressing

The previous chapter ended on `delay` and `force`: a thunk runs at most once, and two identical thunks in one run are the same thunk. `node` extends that idea past the edge of a single run. You wrap a computation in `node`. Its result is content-addressed and written to a store on disk (`~/.pp/store`). The next computation that asks for the same thing finds the answer already there, instead of recomputing it. That next computation might come later in this run, in a separate process, or on another machine sharing the store.

3.1 The node key

What makes two computations "the same thing" is a hash. A node's key is

```
H(code || free-var value-hashes)
```

— the hash of the node's code together with the values of the free variables that code references (this is LAW 20). Nothing else is in the key: not the ambient capabilities, not the config, not the handler stack, not the rest of the environment. Identical code fed identical inputs produces an identical key, and the same key is the same node.

Write the same node twice and pp treats them as one. The first force runs the body and stores the result. The second finds that result by key. A store hit does not replay whatever the body printed, so the side effect happens exactly once:

node-identity.pp

```
# A node is identified by its code and its inputs (LAW 20). Two nodes with
# identical code and free-variable values are the SAME node: the first force
# runs the body and stores the result; the second finds it already in the
# store. A store hit does not replay the body's output (LAW 17), so
# "compiling" prints exactly once.
let first = node { print("compiling"); 6 * 7 }
let second = node { print("compiling"); 6 * 7 }
print(first)
print(second)
```

```
$ pp node-identity.pp
```

```
"compiling"
42
42
```

The key folds in the values of free variables, not the code that computed them. `x` and `y` below are built differently but are the same value. So the two nodes keyed on them are the same node, and the body runs once. A different input means a different key, and runs:

node-freevars.pp

```
# The node key folds in the VALUES of the free variables the body references,
# not the code that produced them (LAW 20). `x` and `y` are computed
# differently but are the same value, so `squared(x)` and `squared(y)` are
# the same node — the body runs once. A different value produces a different
# key, so `squared(9)` runs separately.
def squared(n) { node { print("squaring"); n * n } }
let x = 5
let y = 2 + 3
print(squared(x))
print(squared(y))
print(squared(9))
```

```
$ pp node-freevars.pp
```

```
"squaring"
25
25
"squaring"
81
```

This is why the key excludes the whole environment. Rebind an unrelated global and every node keyed on it is untouched. Change a value a node actually reads, and that node, and only that node, re-keys.

3.2 Caching across runs

Because the store is on disk, node results outlive the process that produced them. A `delay` memoizes within one run; a `node` memoizes across runs. Run a program, exit, then run it again. The second process serves the first's results without repeating the work:

`node-reuse.sh`

```sh
#!/bin/sh
# A node's result is content-addressed in ~/.pp/store, so it survives the
# process that produced it. A second, separate run of the same program finds
# the result already there and does not re-run the body. A throwaway store (a
# temp HOME) keeps this hermetic.
export HOME=$(mktemp -d)
cd "$HOME"

cat > build.pp <<'PP'
def compile() { node { print("compiling greeter.o"); 6 * 7 } }
print(compile())
PP

echo '$ pp build.pp     # first process: the node body runs'
"$PP" build.pp

echo
echo '$ pp build.pp     # second process: served from the store, body skipped'
"$PP" build.pp

rm -rf "$HOME"
```

```
$ pp build.pp     # first process: the node body runs
"compiling greeter.o"
42

$ pp build.pp     # second process: served from the store, body skipped
42
```

The body prints on the first run and is silent on the second. The second process never entered the body. This is the same collapse `force` gives you in memory, made durable: equal computations run once, no matter how many processes ask.

3.3 Pure compute needs no capability

None of these examples were granted anything. `node-identity` and `node-freevars` run with a bare `pp file.pp`, no `--grant`, because arithmetic touches nothing outside itself. Capabilities govern authority over the world, and the chapter on capabilities covers them. A computation that only computes needs none, and caching it needs none either. The moment a node reads the world, the cache has to account for what it saw. Files, configuration, handlers, probes, and child results become trace cells. Ambient subprocesses are rejected. The next chapters cover traces and the narrow provider contract for foreign execution.

4 The store and verifying traces

A node's key decides identity: which computation this is. It does not decide whether a cached result is still good. That is a separate question, and pp answers it in a way that never trusts a clock.

4.1 The store

Everything a node produces lives under `~/.pp/store`, in three content-addressed parts:

store-layout.sh

```sh
#!/bin/sh
# The store has three content-addressed parts. A temp HOME keeps it throwaway.
export HOME=$(mktemp -d)
cd "$HOME"

cat > build.pp <<'PP'
let artifact = node { blob(slurp("input.txt")) }
print(artifact)
PP
echo hello > input.txt

echo '$ pp build.pp --grant fs:$HOME:ro'
"$PP" build.pp --grant "fs:$HOME:ro"

echo
echo '$ ls ~/.pp/store'
ls "$HOME/.pp/store"

rm -rf "$HOME"
```

```
$ pp build.pp --grant fs:$HOME:ro
"5891b5b522d5df086d0ff0b110fbd9d21bb4fc7163af34d08286a2e846f6be03"

$ ls ~/.pp/store
VERSION
blobs
locks
objects
traces
```

- `objects/` — result values, each file named by the hash of the value it holds.
- `traces/` — one file per node key, holding the set of traces recorded for that node (see below).
- `blobs/` — raw bytes ingested by `blob(…)`, named by their hash. The program above turned a file's contents into a blob identity, and the bytes landed here.

(`VERSION` stamps the store format and `locks/` guards concurrent writers. Neither is content you address directly.)

4.2 A hit is a verified trace, not a timestamp

When a node runs, pp records every observation it makes of the world as a `(cell, hash)` pair: each file it reads, each config value, each subprocess tool. That record is a verifying trace: not "this ran at 3pm" but "this is what the node saw."

On the next force, pp does not check a modification time. It re-observes every cell in the trace, and serves the stored result only if every observed hash still matches. A changed input fails the match and forces a recompute. An unchanged input matches and hits. One key holds a set of traces. So a node that ran under two different toolchains or platforms keeps a valid trace for each, and reverting a file re-matches its older trace. A cache hit is therefore always re-checked against the current world. It can never serve stale data, because "stale" means "some observed cell no longer matches", and that is exactly what the hit test checks.

pp checks authority at the same moment. It serves a stored result only if the caller holds capabilities covering every file cell in the trace's read closure (LAW 23b). Reads propagate to enclosing nodes, so the check is transitive: a narrow caller cannot launder a broad read through a cached aggregator. A capability denial is never cached. Authority gates access to a result, it does not rename it.

4.3 pp why

`pp why file.pp` runs the program and reports, to stderr, the fate of every node force: a first-build miss, a verified hit, or a stale trace naming the cell that changed. Here a build reads a file, the same run repeated hits, then you edit the input and the node re-runs:

store-rebuild.sh

```sh
#!/bin/sh
# A cache hit is decided by re-checking what the node observed, not by a
# timestamp. `pp why` reports each node's fate. A temp HOME keeps the store
# throwaway; ROOT is the canonicalized HOME, stripped from output so the
# transcript is machine-independent (the cell ids are absolute paths).
export HOME=$(mktemp -d)
ROOT=$(cd "$HOME" && pwd -P)
cd "$HOME"

cat > build.pp <<'PP'
let greeting = node { slurp("greeting.txt") }
print(greeting)
PP
echo hello > greeting.txt

echo '$ pp why build.pp --grant fs:$HOME:ro    # first build: a miss'
"$PP" why build.pp --grant "fs:$HOME:ro" 2>&1 | sed "s|$ROOT/||g"

echo
echo '$ pp why build.pp --grant fs:$HOME:ro    # input unchanged: a hit'
"$PP" why build.pp --grant "fs:$HOME:ro" 2>&1 | sed "s|$ROOT/||g"

echo
echo goodbye > greeting.txt
echo '$ pp why build.pp --grant fs:$HOME:ro    # input edited: re-run'
"$PP" why build.pp --grant "fs:$HOME:ro" 2>&1 | sed "s|$ROOT/||g"

rm -rf "$HOME"
```

```
$ pp why build.pp --grant fs:$HOME:ro    # first build: a miss
[why] node 3f73f307115b: miss — no stored trace (first build)
"hello\n"

$ pp why build.pp --grant fs:$HOME:ro    # input unchanged: a hit
[why] node 3f73f307115b: hit — ok trace verified (1 cells)
"hello\n"

$ pp why build.pp --grant fs:$HOME:ro    # input edited: re-run
[why] node 3f73f307115b: trace 1/1 stale — file:greeting.txt changed
[why] node 3f73f307115b: miss — no stored trace usable
"goodbye\n"
```

The node key (`f1b010a0e48c` here) is stable: it is a hash of the code, which did not change. What changed is the world the trace was verified against. The first run has nothing to verify against and misses. The second re-observes the one file cell, finds it unchanged, and hits. Editing the file makes that cell's hash no longer match. The stored trace goes stale, no other trace in the set is usable, and the node recomputes.

4.4 Auditing the cache

Two flags let you check the cache instead of trusting it.

`--no-cache` skips cache reads, so every node recomputes, while still writing fresh results and traces. Use it to force a clean build. A node that would otherwise hit reports its miss as `cache reads disabled (--no-cache)`.

`--check` re-runs each missed node's body and compares the new result hash against the stored one. Matching hashes confirm the node is deterministic. A divergence flags the node as volatile and fails the run. This is how you catch a computation that claims to be a pure function of its inputs but is not, the one thing content-addressed caching cannot tolerate.

5 Foreign execution

pp separates computation semantics from execution policy. The language has no compiler, package, container, platform, or Nix form. Libraries build immutable requests as ordinary maps. A trusted host provider decides how to execute one.

`run` is the ambient process effect. It receives command arguments, inherits host behavior, and is always scripting-tier. A node cannot cache it because the request does not account for everything that can affect the result.

`run-closed!` takes immutable tool and input trees, explicit arguments, environment, platform constraints, and selected outputs:

```
run-closed!({
  :tool -> tool-tree,
  :tool-path -> "bin/tool",
  :args -> ["build", "/in/source"],
  :inputs -> source-tree,
  :env -> {},
  :platform -> {"os" -> "linux"},
  :policy -> {:redundancy -> 3},
  :outputs -> ["result"]
})
```

The runtime validates the request and output trees and verifies every blob by hash. It still does not claim that a sandbox makes execution deterministic. Before running a request inside a node, pp asks the installed executor one question: does this exact request account for every semantic input? A trusted provider answers cacheable or scripting-only. pp rejects scripting-only work before the provider executes it.

The bundled Linux provider denies undeclared filesystem, environment, loader, and network access. It still exposes clock, randomness, CPU/kernel behavior, and resource limits, so it honestly classifies every request scripting-only. This makes it useful for portable release actions without making their results silently cacheable.

5.1 Policy belongs to libraries and providers

The classification is the whole core interface. pp does not interpret `:platform` or `:policy`. Policy is optional canonical pp data; its empty default is `{}`. A Nix-like provider can define its schema, verify that a request satisfies it, and return a cacheable classification. A project can use macros to construct those request values. Neither requires an AST form or evaluator change.

An application manifest may provide `:execution-policy` as the default for requests that omit `:policy`. An explicit request policy always wins. The runtime preserves policy data and asks the trusted executor whether the exact request is cacheable; pp libraries define the policy vocabulary.

`stdlib/dune.pp` demonstrates the split. `dune-build(:working-tree, spec)` lets Dune incrementally build an observed development tree and returns a canonical artifact tree. `dune-build(:closed-source, spec)` sends the corresponding immutable request to `run-closed!`. Dune targets, build-directory rules, platform constraints, and output selection stay in the library.

This is the modular boundary: pp supplies lazy nodes, content identity, capabilities, traces, scheduling, and reconciliation. Integrations supply ordinary values and trusted providers.

6 Capabilities and secrets

A pure computation needs no permission: pp will add, compare, and build lists all day without asking. The moment a computation reaches out and touches the world — reads a file, runs a process, makes a request — it needs a capability: an unforgeable token that says it is allowed to. No capability, no effect. This is not a sandboxing layer bolted on the side; it is the only way a world-touching effect ever runs.

Here is a program that tries to read a file with no authority at all:

cap-read.pp

```
# Reading a file requires a filesystem capability. This program is granted
# nothing, so the read is refused before it happens — the check is on
# authority, not on whether the file exists. Observation sigils ($file,
# $env, etc.) are the idiomatic way to read the world; a bare `perform
# read-file` is linted.
print($file("/etc/hostname"))
```

```
$ pp cap-read.pp
```

```
pp: error: slurp: permission denied for /etc/hostname at cap-read.pp:6
```

The read is refused before it happens. The check is on the authority, not on whether the file exists or what it contains — pp never looked. The error names the effect, the missing access, and where the attempt was made.

6.1 Capabilities enter only at the root

There is no expression that creates authority. You cannot write `filesystem("/", :rw)` — `filesystem` is an unbound symbol, not a constructor. The sole mint is the command line: `--grant` hands the program a capability as it starts, and nothing inside the language can manufacture one. Grant the read above a covering filesystem capability and the same effect succeeds:

`cap-grant.sh`

```sh
#!/bin/sh
# The same read-file effect that fails without authority succeeds once the
# root grants a covering capability. Capabilities enter ONLY here, on the
# command line: there is no expression that mints one. A throwaway HOME with
# a file under it keeps the run hermetic; we cd into it and use relative
# paths so nothing machine-specific reaches the output.
export HOME=$(mktemp -d)
mkdir -p "$HOME/etc"
printf 'db1.internal' > "$HOME/etc/hostname"
cd "$HOME"

cat > read-hostname.pp <<'PP'
print(perform read-file("etc/hostname"))
PP

echo '$ pp read-hostname.pp                          # no grant: refused'
"$PP" read-hostname.pp 2>&1

echo
echo '$ pp --grant fs:etc:ro read-hostname.pp     # covering grant: allowed'
"$PP" --grant fs:etc:ro read-hostname.pp 2>&1

rm -rf "$HOME"
```

```
$ pp read-hostname.pp                          # no grant: refused
pp: error: read-file: capability error: no read access for etc/hostname at read-hostname.pp:1

$ pp --grant fs:etc:ro read-hostname.pp     # covering grant: allowed
"db1.internal"
```

A grant is a scheme, a target, and — for the filesystem — a mode:

- `--grant fs:<path>:ro` (or `:rw`, `:wo`) — read, write, or read-write access to a directory and everything under it. Scope matching is by path component on the canonicalized full path, so `fs:/tmp:ro` covers `/tmp/x` but never `/tmpevil`.
- `--grant net:<host>` or `net:<host>:<port>` — network access to a host, and optionally one port; `net:*` wildcards the host.
- `--grant secret:<path>` — a sealed read of the files under a path (below).
- `--grant process` — the authority to run subprocesses.

Pass `--grant` more than once to hold several capabilities at once.

6.2 User code narrows, never widens

Code holds capabilities, passes them around, and attenuates them — but only ever downward. `current-capabilities()` reifies the ambient authority as of the call; it is an observation of the ceiling every `perform` already checks against, not a mint. Two forms narrow it:

- `cap-restrict(cap, scope)` — restricts `cap` to a sub-scope, optionally to a narrower mode, for example `cap-restrict(cap, "src", :ro)`. Asking for a mode wider than `cap` already grants at that scope is a `Capability_error`, never a silent widen.
- `cap-compose(a, b, …)` — unions capabilities the code already holds; it can only ever produce authority already present in its arguments.

`with-caps(cap-expr) { body }` then replaces the ambient authority with `cap-expr` for the extent of `body`. A subset check against the current ambient gates it. So a narrowing composes even when some other binding still lexically holds a broader value. The narrowing is restored when `body` returns, tail-calls, or raises. There is deliberately no union-with-ambient form: the instant capability values exist, unioning with the ambient would be a widening backdoor.

Authority is a ceiling, re-checked at every `perform`. It is never part of a value's identity: widening the grant does not rename or invalidate anything, and narrowing it does not.

6.3 Authority gates the cache, transitively

A cache hit is a message from a past run to this one, so authority has to gate it too — not just live effects. When pp considers serving a node's stored result, it requires the caller's capabilities to cover the transitive read closure of that result: every cell the node read, and recursively every cell its child nodes read. A caller scoped to `src/` is not served a hit on a node whose closure touched `/etc/passwd`. That holds even though the expensive work is already done and sitting in the store. Being handed the result would tell it that the read happened and what it produced. A capability denial is not memoized, so granting the authority later still yields the hit.

This is also why a node may not carry a capability across its own boundary in either direction: a node whose free variable is (or contains) a capability is rejected before it is keyed, and a node whose result is (or contains) one is rejected before it is stored. Authority that rode a cached result out to a narrower caller would defeat the whole check.

6.4 Secrets: sealed cells

A secret is authority-adjacent but a different problem: you want a computation to use a value without that value's bytes leaking into the terminal or, worse, into the content-addressed store, where any later run could read them back. A `secret:` grant handles this. A read covered by `secret:` — and not also by a plain `fs:` grant over the same path — returns a sealed value instead of an ordinary string:

cap-secret.sh

```sh
#!/bin/sh
# A secret is granted with `secret:` instead of `fs:`. A read covered by a
# secret grant (and NOT by a plain fs grant) yields a SEALED value: it prints
# redacted, its bytes are pinned in memory only and never written to the
# content-addressed store, yet a hash of those bytes still rides in the node's
# trace — so rotating the secret invalidates exactly its observers. Hermetic:
# throwaway HOME, a secret file under it, relative paths.
export HOME=$(mktemp -d)
mkdir -p "$HOME/vault"
printf 'hunter2-swordfish' > "$HOME/vault/token"
cd "$HOME"

# (1) A sealed value prints redacted — the bytes never reach the terminal.
cat > show.pp <<'PP'
print(slurp("vault/token"))
PP
echo '$ pp --grant secret:vault show.pp'
"$PP" --grant secret:vault show.pp 2>&1

# (2) Derive from the secret inside a node (its length), which populates the
#     store, then scan the whole store for the plaintext.
echo
cat > derive.pp <<'PP'
print(force(node { string-length(unseal(slurp("vault/token"))) }))
PP
echo '$ pp --grant secret:vault derive.pp     # length of the secret'
"$PP" --grant secret:vault derive.pp 2>&1

echo
echo '$ grep -r hunter2-swordfish ~/.pp/store  # the bytes are not there'
if grep -rq 'hunter2-swordfish' "$HOME/.pp/store" 2>/dev/null; then
  echo 'FOUND (leak)'
else
  echo '(no match — secret bytes absent from the store)'
fi

rm -rf "$HOME"
```

```
$ pp --grant secret:vault show.pp
#<sealed>

$ pp --grant secret:vault derive.pp     # length of the secret
17

$ grep -r hunter2-swordfish ~/.pp/store  # the bytes are not there
(no match — secret bytes absent from the store)
```

Three things hold at once. The sealed value prints redacted as `#<sealed>`, so it cannot leak by being printed. Its bytes are pinned in memory only and are never written to the store, so a scan of the whole store never finds them — the `grep` above comes up empty even though a node ran and populated the store. And a hash of those bytes does ride in the node's trace, so the cache stays correct: rotate the secret and exactly the nodes that observed it are invalidated, while their siblings keep hitting.

Deriving from a secret inside a node — the length, above — produces ordinary data, because the derived value is no longer the secret. The one explicit way to turn a sealed value back into a plain string is `unseal(v)`; there is no implicit dataflow tainting. Unsealing is a deliberate, greppable boundary, so the confidentiality of a secret ends exactly where the program says it does. If a path is covered by both a `secret:` and an `fs:` grant, the ordinary filesystem behaviour wins: the deployment that granted plain access is saying "not secret here".

7 Modules and islands

A module is a block of code whose definitions become a value. That is the whole idea. Packaging is not a separate mechanism layered over the language. It is what happens when you take the names a block defined and hand them back as something you can bind, pass, and import. Islands extend the same value across machines by pinning it to a content hash.

7.1 Modules

`module { … }` evaluates its body in a fresh scope and produces a value carrying the names it defined. `import(m)` forces that value and merges those names into the current scope:

mod-module.pp

```
# module { ... } evaluates its body in a fresh scope and produces a value whose
# fields are the names it defined. import(...) forces that value and merges
# those names into the current scope. Here the module is bound to `geo`, then
# imported, so `area` and `pi` become callable.
let (geo = module {
  let pi = 3.14159
  def area(r) { pi * (r * r) }
}) {
  import(geo)
  print(area(10))
}
```

```
$ pp mod-module.pp
```

```
314.159
```

The module's body sees only itself, not the scope around it, so a module is a sealed unit: what escapes is exactly what it defined, and only once you import it. You can hold a module value, pass it to a function, or import it conditionally — it is an ordinary value until the moment you pull its names in.

7.2 Loading from files

Two forms bring a file's definitions into a program, and they differ in whether the file gets its own scope:

- `load("f.pp")` evaluates the file's forms in the current scope — its definitions merge in directly, as if you had typed them here.
- `load-module("f.pp")` evaluates the file isolated, in its own fresh scope, and returns its exports as a module value — which you then `import`. The file's internal scope never leaks into yours.

mod-load-module.sh

```sh
#!/bin/sh
# Two ways to bring a file's definitions in. `load` MERGES a file into the
# current scope (its defs become directly visible). `load-module` loads the
# same file ISOLATED and returns its exports as a value, which you then
# `import` — so the file's own scope never leaks in. Hermetic throwaway HOME.
export HOME=$(mktemp -d)
cd "$HOME"

cat > mathlib.pp <<'PP'
def square(x) { x * x }
def cube(x) { x * square(x) }
PP

cat > use-load.pp <<'PP'
# load: mathlib's defs merge into this scope.
load("mathlib.pp")
print(square(9))
PP
echo '$ pp use-load.pp          # load merges definitions in'
"$PP" use-load.pp 2>&1

echo
cat > use-load-module.pp <<'PP'
# load-module: mathlib loads isolated; its exports come back as a value.
let (m = load-module("mathlib.pp")) {
  import(m)
  print(cube(3))
}
PP
echo '$ pp use-load-module.pp   # load-module returns exports to import'
"$PP" use-load-module.pp 2>&1

rm -rf "$HOME"
```

```
$ pp use-load.pp          # load merges definitions in
81

$ pp use-load-module.pp   # load-module returns exports to import
27
```

Reach for `load` when a file is a fragment of the current program, and `load-module` when it is a self-contained unit you want to keep at arm's length. Either way, the read is the loader's own — it runs under the interpreter's runtime authority, bounded to your source roots, and is exempt from the capability accounting of the previous chapter. Editing a loaded file still invalidates the nodes that loaded it: the load is recorded in their traces even though it costs no user capability.

7.3 Islands

An island is a module that lives elsewhere — another directory, a git repository, a URL — referenced by URI and pinned inline by the content hash of its source tree:

```
island("file:./lib", "e5d7b0f8…")
island("github:owner/repo#main", "a1b2c3…")
```

The pin is part of the code. Because it folds into the expression's hash, island identity is structural: there is no lockfile and no hidden resolution step — a pinned island form denotes the same bytes wherever you paste it. The ref after `#` lives in the URI and matters only when fetching; the pin argument is always the 64-hex content hash of the tree.

Resolution never touches the network. The pin names an immutable tree in the island cache under `~/.pp/islands/src/<pin>/`, whose `entry.pp` is the module root. pp verifies the cached tree against the pin on every resolve. A mismatch is a hard error, not a silent reuse. An unpinned island form is a hard error too, naming the fix. Deriving the pin, and fetching a `git:` or `github:` tree the first time, is the one impure step, and it is opt-in. `pp --update` re-resolves each island and rewrites the pins in your source, while `git:`/`github:` fetching happens only under `--fetch-islands`. With neither enabled, evaluation stays hermetic.

The workflow, end to end, on a local `file:` island:

mod-island.sh

```sh
#!/bin/sh
# An island is a module referenced by URI and pinned INLINE by the content
# hash of its source tree. Resolution never touches the network: the pin names
# an immutable tree in the island cache. An unpinned form is a hard error that
# names the fix; `pp --update` derives the pin and writes it into the source;
# the pinned form then evaluates the tree's entry.pp as a module. Hermetic
# throwaway HOME (which also holds the island cache).
export HOME=$(mktemp -d)
mkdir -p "$HOME/lib"
printf 'def greet(who) { string-append("hello, ", who) }\n' > "$HOME/lib/entry.pp"
cd "$HOME"

cat > app.pp <<'PP'
let (m = island("file:./lib")) {
  import(m)
  print(greet("world"))
}
PP

echo '$ pp island-pins app.pp   # the form is unpinned'
"$PP" island-pins app.pp 2>&1

echo
echo '$ pp --update app.pp      # derive the pin and write it into the source'
"$PP" --update app.pp 1>/dev/null

echo
echo '$ cat app.pp              # the pin is now part of the code'
cat app.pp

echo
echo '$ pp app.pp               # resolve the pinned tree and run it'
"$PP" app.pp 2>&1

rm -rf "$HOME"
```

```
$ pp island-pins app.pp   # the form is unpinned
file:./lib  (unpinned)

$ pp --update app.pp      # derive the pin and write it into the source
[update] app.pp: 1 pin(s) updated, 0 skipped

$ cat app.pp              # the pin is now part of the code
let (m = island("file:./lib", "0d4042cbc451a50074019c14ebdd52144be6ba610d60d470159b1f1a09954cf7")) {
  import(m)
  print(greet("world"))
}

$ pp app.pp               # resolve the pinned tree and run it
"hello, world"
```

`pp island-pins` reports the forms and whether each is unpinned, cached, or tampered; `pp --update` fills in a missing pin rather than making you compute the hash by hand, and refuses to touch a form it cannot rewrite unambiguously rather than half-writing your file. Once the pin is in place, the island is just another module: `import` its value and call what it exports.

8 Domains and the reconciler

A node computes a value. But a build has to change the world: write files, start processes. pp will not let you do that by mutating it directly. Instead you compute a desired state: a pure, hashable value that says what the world should contain — `{path → content}`, `{service → spec}`. A domain observes the world, diffs it against your desired value, and applies the minimal change to converge them. This is React's contract verbatim. You never touch the DOM. You return the desired DOM and the reconciler applies the diff. Here the world is the filesystem and the process table.

The payoff is that convergence follows from the value, not from a script you maintain. Your desired state is a pure function of input cells. So pp caches it, re-derives it cheaply when an input changes, and re-observes reality rather than trusting a state file. Drift is just a difference the next pass erases — a file someone edited by hand, a service that died.

8.1 A domain is observe / diff / apply

A domain is three ordinary pp functions registered over a namespace of cells:

- observe `: () → value` reads the current world. It runs fresh every pass, never cached.
- diff `: (observed, desired) → plan` computes what to change. It is pure: it runs under an empty capability set and cannot touch the world. Its result is cached, keyed on the diff code plus the observed and desired values.
- apply `: plan → nil` performs the plan's changes, under the domain's own write capability.

Core wraps every domain's apply in one discipline: a journalled intent/done bracket, atomic writes, and verify-after-write. Core re-observes and re-diffs once the apply returns. A plan that still has work left is a hard error. A domain is the single writer for its namespace. So there are no write-write races, and your code needs no ordering discipline. One rule enforces this: stratification. The desired state may not read the domain's own cells, or reconcile would loop forever. Reading your output tree to decide your output tree is refused.

The trusted mechanics that actually touch the world are a small set of core primitives: atomic temp-file-plus-`rename`, `fork`/exec/reap, a per-domain key-value store (`tree-observe`, `materialize-file`, `remove-file`, `proc-spawn`, and the like). Everything else is a pp library. The filesystem and process domains ship as `stdlib/domain-fs.pp` and `stdlib/domain-proc.pp`. This is real pp source you can read. It holds all the policy over the core-enforced protocol: what counts as a create versus an update, when to restart a service. There is no privileged reconciler engine hidden in the runtime. A domain is a library.

8.2 Reconciling a filesystem

`pp --reconcile ROOT prog.pp` auto-loads the fs domain and registers it with a write capability narrowed to `ROOT`. It takes the program's final value as the desired state of the tree under `ROOT`: a canonical tree value. File entries carry a mode and blob identity; directory entries make parents explicit. It diffs that against the real directory by content hash and materializes missing and changed files atomically. It also deletes files under `ROOT` that the map does not mention. The domain is the single writer, and the write grant is your consent to that authority. An fs write grant over `ROOT` is required. Without it, nothing is written.

The transcript below reconciles a two-file desired state into a fresh root, so both files appear. It then introduces drift by hand, deleting one file and editing another, and reconciles again. The second pass reports exactly one create and one update, and the tree is restored. The summary line pp prints names the counts per kind. Its absolute root path is filtered to `ROOT` so the output is machine-independent.

domain-reconcile.sh

```sh
#!/bin/sh
# A domain converges the world to a DESIRED state you return as a value. The
# fs domain (stdlib/domain-fs.pp) takes a canonical tree value and
# makes a directory tree match it: creating, updating, and deleting files.
# Hermetic: a throwaway HOME (so the store and journal are fresh) and a root
# under it. The reconcile summary's absolute root path is filtered to ROOT.
export HOME=$(mktemp -d)
ROOT="$HOME/site"

cat > "$HOME/site.pp" <<'PP'
{:tree -> {
  "index.html" -> {:kind -> :file, :mode -> 420, :blob -> blob("<h1>pp</h1>\n")},
  "conf" -> {:kind -> :directory, :mode -> 493},
  "conf/app.txt" -> {:kind -> :file, :mode -> 420, :blob -> blob("mode=prod\n")}
}}
PP

filter() { sed "s#root=[^ ]*#root=ROOT#"; }

echo '$ pp --grant fs:ROOT:rw --reconcile ROOT site.pp   # first pass: create'
"$PP" --grant "fs:$ROOT:rw" --reconcile "$ROOT" "$HOME/site.pp" 2>&1 | filter
( cd "$ROOT" && find . -type f | sort )
echo "conf/app.txt: $(cat "$ROOT/conf/app.txt")"

echo
echo '# Introduce drift: delete one file, corrupt another.'
rm "$ROOT/index.html"
echo 'mode=DEV' > "$ROOT/conf/app.txt"

echo '$ pp --grant fs:ROOT:rw --reconcile ROOT site.pp   # second pass: restore'
"$PP" --grant "fs:$ROOT:rw" --reconcile "$ROOT" "$HOME/site.pp" 2>&1 | filter
( cd "$ROOT" && find . -type f | sort )
echo "conf/app.txt: $(cat "$ROOT/conf/app.txt")"

rm -rf "$HOME"
```

```
$ pp --grant fs:ROOT:rw --reconcile ROOT site.pp   # first pass: create
[reconcile:fs] root=ROOT create=2 update=0 delete=0
./conf/app.txt
./index.html
conf/app.txt: mode=prod

# Introduce drift: delete one file, corrupt another.
$ pp --grant fs:ROOT:rw --reconcile ROOT site.pp   # second pass: restore
[reconcile:fs] root=ROOT create=1 update=1 delete=0
./conf/app.txt
./index.html
conf/app.txt: mode=prod
```

Nothing in `site.pp` describes how to converge. There is no “if missing, create” logic. It states the desired tree, and the domain works out the difference. Reverting `conf/app.txt` and restoring the deleted `index.html` are the same mechanism, driven entirely by the diff.

File contents are raw blob identities in the content-addressed store. A blob identity diffs without loading its bytes. So `rm -rf` on the tree restores from the store with zero tool re-runs when the desired-state nodes hit.

8.3 Watching, and other domains

`pp --watch --reconcile ROOT prog.pp` runs the program, reconciles, then polls the observed cells and re-runs on any change. This is a controller loop. Every registered domain is re-observed and re-applied on each tick, whichever cell changed. This is cheap when nothing moved, because the plan cache turns a no-op pass into a hit. An externally deleted file or a drifted config is caught within one poll interval.

The process domain is the same protocol over a different world. `pp --supervise prog.pp` (usually with `--watch`) takes a `{service-name → spec}` map and keeps the process table matching it. It starts missing services, stops removed ones, and restarts a service whose spec changed. It also restarts one killed out from under it, within a poll interval. It requires `--grant process` and refuses stratification on its own `proc:` cells, exactly as the fs domain does. A from-scratch third-party domain is anything with an observe/diff/apply triple registered via `register-domain`. It gets the same journal bracket, plan cache, verify-after-write, and stratification for free. The protocol is generic, not filesystem-shaped.

Load `stdlib/domain.pp` for composition helpers such as `domain(spec)`, `probe(name, observe, cap)`, and `register-domains(domains)`. These are ordinary pp functions and do not bypass the generic lifecycle.

8.4 Fenced effects

Convergence covers only idempotent change: applying a desired state twice is the same as applying it once. Some actions are not like that, such as sending an email or charging a card. Those may not appear in a node at all. A node is cache-replayable and must never carry an irreversible action. The scripting form `fenced(KIND, SPEC)` registers such an action for the reconciler to run after all convergent work, at most once per pass. It carries an intent/done journal. So a crash mid-action is recovered by policy (`--fenced-policy retry | abort | ask`) rather than by a silent retry that might double-charge. The desired-state law tames the convergent world. Fenced effects are the named carve-out for the part that cannot be converged.

9 Distribution

Here is the thesis this whole manual has been building toward: there is no local/remote distinction at the language surface. There is no `remote-eval`, no placement annotation, no node-pinning form, and there never will be. `force` is the only execution primitive. Where a force runs is a scheduler decision, chosen with a flag, never written into the program. A program is byte-identical whether it runs on one core, eight, or a cluster. Microservices exist because no mainstream language lets you say "evaluate this elsewhere and flow the result back". The moment location becomes syntax, every caller hard-codes topology, and you have rebuilt the deployment boundary inside the language.

`--schedule serial | parallel:N | race:N | remote:<member>` selects the policy. `parallel:N` forks workers across cores. `race:N` runs N redundant copies and takes the first. `remote:<member>` ships the work to another machine. These are the same feature at different fan-out. "Run on N machines and take the first" is a handler swap, not an infrastructure project. The scheduler is result-transparent: it changes only where and when work runs, never the answer. So it appears in no cache key and no trace. `pp --check` proves this. It re-runs any non-serial policy forced serial against the same store and fails on any hash mismatch.

Libraries can select the same handlers through `stdlib/runtime.pp`. They can also provide a pure custom scheduler policy over data-only job descriptors. The policy returns a mode and a complete partition of job indexes; the runtime still owns execution, cancellation, and remote transport.

9.1 Identity is location-independent

Location can be a scheduler's business, not the program's, because identity is a content hash. A node's key is a hash of its code and its inputs' values, never of where or when it ran. Two machines that evaluate the same node therefore compute the same key and a byte-identical result, with no coordination. A result is then moved by that hash. It is published into a shared store and pulled by name, then re-hashed on receipt so a corrupted or tampered artifact is refused. The content address is the integrity check.

The transcript below uses two throwaway `HOME`s as two machines, each with its own `~/.pp` store, and a shared directory as the transport. Machine A builds a node. Machine B, given the same program, computes the identical node key without ever talking to A. Then A publishes the result object by hash and B pulls it, byte-identical. Flipping one byte in transit makes B reject it.

dist-by-hash.sh

```sh
#!/bin/sh
# Distribution in pp is not a language feature: identity is a content hash, so
# a computation has the SAME name on every machine, and a result is moved by
# that hash. Two throwaway HOMEs stand in for two machines (each has its own
# ~/.pp store); a shared directory is the transport (a plain local-dir
# loopback — no network transport ships yet (see docs/THREAT-MODEL-cluster.md).
# Content hashes are deterministic, so they are reproducible output.
export HOME=$(mktemp -d)
A="$HOME/machine-a"; B="$HOME/machine-b"; SHARED="$HOME/shared"
mkdir -p "$A" "$B" "$SHARED"

cat > "$HOME/node.pp" <<'PP'
print(force(node {
  perform log("building greeter.o")
  6 * 7
}))
PP

echo '$ pp node.pp                 # on machine A: the node body runs'
HOME="$A" "$PP" "$HOME/node.pp" 2>&1
KEY=$(ls "$A/.pp/store/traces")
RH=$(grep -oE '"[0-9a-f]{64,}"' "$A/.pp/store/traces/$KEY" | head -1 | tr -d '"')
echo "node key:    $KEY"
echo "result hash: $RH"

echo
echo '$ pp node.pp                 # on machine B, independently: SAME node key'
HOME="$B" "$PP" "$HOME/node.pp" 2>&1
KEY_B=$(ls "$B/.pp/store/traces")
[ "$KEY_B" = "$KEY" ] && echo "machine B computed the same key: yes"

echo
echo '# A publishes the result object by hash; B pulls it by that hash.'
echo '$ pp --transport-push object <hash> shared    # on A'
HOME="$A" "$PP" --transport-push object "$RH" "$SHARED" >/dev/null 2>&1
echo '$ pp --transport-pull object <hash> shared    # on B, re-hash-verified'
HOME="$B" "$PP" --transport-pull object "$RH" "$SHARED" 2>&1
diff -q "$A/.pp/store/objects/$RH" "$B/.pp/store/objects/$RH" >/dev/null \
  && echo "byte-identical across the transport: yes"

echo
echo '# A tampered object is refused on receipt: the hash IS the integrity check.'
printf 'X' | dd of="$SHARED/objects/$RH" bs=1 seek=3 count=1 conv=notrunc 2>/dev/null
echo '$ pp --transport-pull object <hash> shared    # after a byte flips in transit'
HOME="$B" "$PP" --transport-pull object "$RH" "$SHARED" 2>&1

rm -rf "$HOME"
```

```
$ pp node.pp                 # on machine A: the node body runs
[info] building greeter.o
42
node key:    2e439de4fe0f78cca7d94cefffde961feb12d60279645006f536c2768d1fb6dd
result hash: d98fba09338648079975ad61b2a3ca43cb3ddf4e2c36c7724f66aab333224012

$ pp node.pp                 # on machine B, independently: SAME node key
[info] building greeter.o
42
machine B computed the same key: yes

# A publishes the result object by hash; B pulls it by that hash.
$ pp --transport-push object <hash> shared    # on A
$ pp --transport-pull object <hash> shared    # on B, re-hash-verified
byte-identical across the transport: yes
```

The two machines never agreed on a key. They derived the same one, because the key is a function of the code and nothing else. That is what makes a build computed “here” usable “there”. The result is addressed by what it is, not by where it was produced.

9.2 Transport boundary

The by-hash sync, re-hash-on-receipt, signed-token authority checks, and remote placement are implemented and tested over a local-directory transport. Two separate `~/.pp` stores stand in for two machines. No SSH implementation or generic transport plugin ships today. A remote provider can carry the same canonical artifacts and signed requests; integrity and authority remain runtime checks rather than promises made by the connection.

9.3 Host-qualified identity

A running process is not a pure value. Here the “same identity everywhere” story inverts in exactly the right way: a process on host B is a different cell than the same process on host A. Host-qualified identity names world state, “the greeter running on B”, not a location you dispatch to. The desired-state map from the previous chapter generalizes one level for this, from `{domain → desired}` to `{host → {domain → desired}}`. `--member-name B` selects host B’s slice and hands it to the unchanged reconciler. A member is then just `pp --watch --supervise --member-name B` on its own slice: the single-machine supervisor, verbatim, fed a slice of a cluster-wide value. Naming a host absent from the map is a hard error, not a silent no-op.

Membership is ambient configuration: which machines exist, and where their stores are. It is read from `~/.pp/cluster/members` or `$PP_CLUSTER_MEMBERS`. It is deliberately not a capability. An address is not an authority ceiling, the same distinction the language draws everywhere between location and permission. Authority to act across the cluster travels separately, as a signed token minted from the same `--grant` grammar you use locally.

9.4 Trace merge, audit, and growth

Because a result carries its verifying trace, syncing a result syncs its evidence. A machine that pulls a node’s result also pulls the `(cell, hash)` observations that justify it. It re-verifies them against its own world before serving a hit. The authority checks survive the crossing. A hit served across the wire is gated on the requesting token’s authority over the trace’s read closure, exactly as a local hit is gated on the caller’s. A narrow caller cannot launder a broad read through a cached aggregator just because the bytes arrived over a network. And `pp why`, run on a machine that pulled a trace, redacts any cell the local caller is not authorized to see, identically to a purely local run. Audit redacts. It never lies.

A shared store only grows, so growth is bounded explicitly. Each successful pass records its desired object and forced node keys as a wanted root. `pp gc` walks the same durable child, result, and tree-blob edges used by validation and stabilization, then sweeps only unreachable objects, traces, and blobs. It is never automatic. A grace period protects concurrent writes, and a changed root manifest stops deletion.

10 The engine

pp runs every program through a single tree-walking evaluator. It interprets the AST directly — small, obvious, and the reference for what every form means.

Correctness is checked by a metamorphic fuzzer: it generates random programs, applies semantics-preserving transforms to produce a twin (do-wrap, let-identity, eta-identity), and asserts the original and the twin produce identical output. It also runs a reader round-trip gate: every generated program is printed and re-read, and the ASTs must be structurally equal with identical hashes. Any mismatch is shrunk to a minimal repro and written to a failure directory.

10.1 The metamorphic fuzzer

The fuzzer (`tools/fuzz.ml`) is the project's single most valuable correctness asset. It generates random pp programs under two grammars. `core` covers literals, control flow, functions, definitions, the arithmetic and collection builtins, and the stdlib list functions — any non-PASS on `core` is a real bug and fails CI. `full` adds the harder surface: type annotations, modules, effects and handlers, deep recursion, quotation, and `defmacro`.

```
dune exec ./tools/fuzz.exe -- --grammar core --count 2000
dune exec ./tools/fuzz.exe -- --grammar full --count 2000
```

The fuzzer shells out to the interpreter, defaulting to `bin/pp`. The generator is fully deterministic: program i under a given seed is always the same program, so any finding reproduces exactly.

10.2 The expected-output suite

The test suite (`dune runtest`) runs every `tests/NNN-*.pp` program and diffs its stdout against a blessed `tests/NNN-*.pp.expected` file. A missing `.expected` file is a failure — it means the test hasn't been blessed yet. This catches print- and effect-ordering changes that the fuzzer's twin oracle might not.

The suite also runs shell suites (`tests/*.sh`) for multi-process scenarios: store persistence across runs, watch loops, cluster members, and crash injection.

10.3 Kernel properties

`src/app/kernel_props.ml` runs QuickCheck-style property sweeps: hash injectivity, the quote round-trip, and the print round-trip over random ASTs and values. The generators match exhaustively on constructor tags, so a new AST or capability kind breaks the build until it is generated and covered.

11 Language reference

This appendix gives the full detail the language chapter summarized. Each section covers one construct and shows it running. pp is a Lisp-1: functions and variables share one namespace, and evaluation is inner-first and strict — a form's arguments are values by the time it runs.

The special forms — the syntax the reader treats specially, rather than ordinary function calls — are: `if`, `do`, `let`, `let*`, `fn`, `def`, `node`, `match`, `quote`, `quasiquote` with `unquote` and `splice`, `and`, `or`, `delay`, `force`, `perform`, `with-handler`, `with-caps`, `with-config`, `config`, `assert`, `module`, `import`, `load`, `load-module`, `island`, `reconcile`, `try`, and the `:` type annotation. `defmacro` is recognized structurally at the top level; `needs` modifies a named `node` definition. Everything else is a function. In the s-expression surface, named node forms are represented as `defnode`, and `splice` is represented as `unquote-splicing`.

The brace surface also supplies infix operators, pipelines (`|>`), f-strings, spread (`...`), observation sigils (`$file`, `$env`, `$glob`, `$probe`, `$secret`, `$config`), and the `with { caps: ..., config: ..., handlers: ... }`
`{ ... }` clause form. Their exact lowerings are listed in the syntax appendix of `SPEC.md`.

11.1 Value types

Integers, floats, strings, the booleans `true` and `false`, `nil`, keywords (`:name`), and quoted symbols (`quote { name }`) are the atoms. `print` shows strings quoted and prints one value per call.

`ref-atoms.pp`

```
# Integers, floats, strings, booleans, nil, keywords, and quoted symbols.
# `print` shows strings quoted; everything else prints as it reads.
print(42)
print(3.14)
print("text")
print(true)
print(false)
print(nil)
print(:keyword)
print(quote { sym })
```

```
$ pp ref-atoms.pp
```

```
42
3.14
"text"
true
false
nil
:keyword
sym
```

The collections are lists, vectors, maps, and sets, each with its own printed form.

`ref-collections.pp`

```
# Lists, vectors, maps, and sets — each with its own literal syntax.
print([1, 2, 3])
print(vec[1, 2, 3])
print({:a -> 1, :b -> 2})
print(hash-set(1, 2, 3))
```

```
$ pp ref-collections.pp
```

```
(1 2 3)
[1 2 3]
{:a 1, :b 2}
#{1 2 3}
```

11.2 Arithmetic and comparison

`+` and `*` are variadic. `-` and `/` take exactly two arguments. `mod` is the integer remainder.

ref-arith.pp

```
# `+` and `*` are variadic; `-` and `/` take exactly two arguments; `mod`
# is the remainder. Comparisons chain left to right: `<(1, 2, 3)` is
# `1 < 2 AND 2 < 3`.
print(+(1, 2, 3))
print(10 - 3)
print(*(2, 3, 4))
print(20 / 4)
print(17 mod 5)
```

```
$ pp ref-arith.pp
```

```
6
7
24
5
2
```

Comparisons are variadic and chain left to right — `<(1, 2, 3)` means `1 < 2` and `2 < 3`.

ref-compare.pp

```
# Comparisons are variadic and chain left to right.
print(2 = 2)
print(<(1, 2, 3))
print(>(3, 2, 1))
print(<=(1, 1, 2))
print(3 >= 3)
```

```
$ pp ref-compare.pp
```

```
true
true
true
true
true
```

11.3 Booleans and conditionals

Only `nil` and `false` are falsy; every other value, including `0`, is truthy. `and` and `or` short-circuit and return the deciding value rather than a boolean.

ref-bool.pp

```
# Only nil and false are falsy; everything else (including 0) is truthy.
# `and`/`or` short-circuit and return the deciding value, not a boolean.
# Flat `else if` chains keep multi-way branching clean.
print(if 1 { if 2 { 3 } else { false } } else { false })
print(if false { true } else if nil { true } else { 5 })
print(if true { false } else { false })
print(not(nil))
print(not(0))
```

```
$ pp ref-bool.pp
```

```
3
5
false
true
false
```

`if` is an expression that evaluates exactly one branch. The untaken branch never runs — no effects, no errors.

ref-if.pp

```
# `if` is an expression and evaluates exactly one branch — the untaken one
# never runs, so the error here is never raised.
print(if 5 > 0 { "pos" } else { "neg" })
print(if false { error("never") } else { "safe" })
```

```
$ pp ref-if.pp
```

```
"pos"
"safe"
```

11.4 Bindings

`let` bindings are mutual: every binding is visible in every right-hand side and in the body, regardless of the order written.

ref-let.pp

```
# `let` bindings are mutual: every binding sees every other, whatever the
# textual order. One flat `let` with all bindings is the idiomatic style.
print(let (y = x + 1, x = 1) { y })
```

```
$ pp ref-let.pp
```

```
2
```

`let*` is the sequential form. Each binding sees only the ones above it, so a name may shadow itself.

ref-letstar.pp

```
# `let*` is sequential: each binding sees only the ones written above it,
# so a name can shadow itself.
print(let* (x = 1, x = x + 1) { x })
```

```
$ pp ref-letstar.pp
```

```
2
```

At the top level, names are predeclared across the scope. A function definition can refer to a later definition; a value definition still evaluates its value at its statement, so a later value is not available until that statement runs.

ref-def.pp

```
# At the top level, names are predeclared across the scope. Function definitions
# can refer forward; value definitions still evaluate at their statement.
let answer = 6 * 7
print(answer)

def square(x) { x * x }
print(square(7))
```

```
$ pp ref-def.pp
```

```
42
49
```

11.5 Functions

`fn` produces an anonymous function; `def` with a list head names one. There is no loop keyword — recursion is the loop, and tail calls run in constant stack.

ref-fn.pp

```
# `fn` makes an anonymous function; recursion is the only loop.
print((fn(x) { x + 1 })(41))

def factorial(n) {
  if n = 0 { 1 } else { n * factorial(n - 1) }
}

print(factorial(5))
```

```
$ pp ref-fn.pp
```

```
42
120
```

11.6 Nodes

`node { ... }` creates a persistent, content-addressed computation. Its body runs when forced, and its result can be reused across processes. A named, zero-argument node uses `node build { ... }` and binds a node thunk. A parameterized `node compile(src) { ... }` is the brace spelling of the s-expression `defnode` form; applied named-node keying remains a documented LAW 6 follow-up, so the current persistent keying guarantee is for node thunks, not an argument-keyed application cache. `needs` narrows the capabilities available to the node body, for example `node read(path) needs fs.read(path) { ... }`.

The node key contains the body and the values of the free variables it reads; ambient capabilities, handlers, and config are not identity inputs.

node-identity.pp

```
# A node is identified by its code and its inputs (LAW 20). Two nodes with
# identical code and free-variable values are the SAME node: the first force
# runs the body and stores the result; the second finds it already in the
# store. A store hit does not replay the body's output (LAW 17), so
# "compiling" prints exactly once.
let first = node { print("compiling"); 6 * 7 }
let second = node { print("compiling"); 6 * 7 }
print(first)
print(second)
```

```
$ pp node-identity.pp
```

```
"compiling"
42
42
```

11.7 Sequencing

`do` forces each form in order for its effects and returns the value of the last.

ref-do.pp

```
# `do` runs each form in order for its effects and returns the last value.
print(do {
  print("step 1")
  print("step 2")
  99
})
```

```
$ pp ref-do.pp
```

```
"step 1"
"step 2"
99
```

11.8 Type annotations

Annotations are optional gradual claims, written with `:`, and checked when the function's body runs — not before. A well-typed call passes through unchanged.

ref-types.pp

```
# Type annotations are optional and checked when the body runs. A well-typed
# call passes through unchanged.
def increment(n: int): int { n + 1 }
print(increment(41))
```

```
$ pp ref-types.pp
```

```
42
```

A mismatch names the offending value and its source location.

ref-type-error.pp

```
# A type annotation mismatch names the offending value and location.
def increment(n: int): int { n + 1 }
increment("oops")
```

```
$ pp ref-type-error.pp
```

```
pp: error: type mismatch: expected int, got "oops" at ref-type-error.pp:2
```

11.9 Quotation

Braces are pp's surface syntax; s-expressions are its AST. `quote { ... }` is the bridge: it turns the one form inside into that AST, as data. This is the same position Elixir takes — homoiconicity lives at the AST layer, not the surface. The brace text you type is not itself a data structure, but the tree it reads to is, and `quote` hands you that tree. Quotation is total: any form the reader accepts, `quote` turns into data.

Quasiquote builds structure with holes. The body of `quasiquote { ... }` is a template written in ordinary brace syntax, denoting the s-expression data it reads to; `unquote(e)` fills one hole with a computed value, `splice(e)` splices a list into a list position.

ref-quote.pp

```
# Braces are the surface; s-expressions are the AST. `quote { ... }` turns
# the form inside into that AST, as data. `quasiquote { ... }` is a template:
# `unquote(e)` fills one hole, `splice(e)` splices a list into a list position.
print(quote { if a { b } else { c } })
print(quasiquote { f(1, unquote(1 + 1), splice(list(3, 4))) })
```

```
$ pp ref-quote.pp
```

```
(if a b c)
(f 1 2 3 4)
```

11.10 Laziness

`delay` suspends a computation; `force` runs it and memoizes the result, so the body runs at most once however many times it is forced. `force` is the identity on non-thunks.

ref-delay.pp

```
# `delay` suspends a computation; `force` runs it and memoizes the result,
# so the body runs at most once however many times it is forced.
let (t = delay(do {
  print("run once")
  42
})) {
  print(force(t))
  print(force(t))
}
```

```
$ pp ref-delay.pp
```

```
"run once"
42
42
```

11.11 Effects and config

`perform` dispatches an effect to the nearest handler installed by `with-handler`. The clause form `with { handlers: { :name -> fn(...) { ... } } } { ... }` installs the same handler map. Handling an effect yourself needs no capability. Built-in world effects include `read-file`, `write-file`, `run`, `run-closed!`, `http-get`, `http-post`, `log`, `tree-observe`, the process effects, and the domain-state effects; user handlers may define additional effect names. Ambient process and network effects are scripting-tier. `run-closed!` is cacheable only when its installed provider classifies the exact request as such.

ref-perform.pp

```
# `perform` dispatches to the nearest handler installed by `with-handler`.
# The `handlers:` clause takes a map of keyword -> function pairs.
with { handlers: { :double -> fn(x) { x * 2 } } } {
  print(perform double(21))
}
```

```
$ pp ref-perform.pp
```

```
42
```

Config is ambient, dynamically-scoped data, distinct from capabilities. `with-config(map) { ... }` installs a temporary config frame, and `config` reads a key with an optional default for when it is absent. `with-caps(cap) { ... }` narrows the current capability set for the body's dynamic extent; it cannot widen authority.

ref-config.pp

```
# Config is ambient, dynamically-scoped data. `$config(key)` reads a key,
# with an optional default when the key is absent. It records a `config:`
# trace cell so a node that observed config recomputes when it changes.
print(with-config({:host -> "db1", :port -> 5432}) { $config(:host) })
print(with-config({:host -> "db1"}) { $config(:timeout, 30) })
```

```
$ pp ref-config.pp
```

```
"db1"
30
```

11.12 Assertions

`assert(condition)` raises a located error when the condition is false. `assert(condition, message)` uses the supplied message. It is syntax sugar for an `if` that returns `nil` on success and calls `error` on failure.

ref-assert.pp

```
# `assert` succeeds silently when its condition is true and returns nil.
assert(1 = 1)
print("ok")
```

```
$ pp ref-assert.pp
```

```
"ok"
```

11.13 Macros

`defmacro` defines a macro: it receives its arguments as unevaluated forms — s-expression data, the same trees `quote` yields — and returns a new form, expanded before the evaluator sees it. You write the macro in braces; it consumes and produces the AST. A `quasiquote { ... }` template is the usual way to assemble the expansion: ordinary brace syntax with `unquote(e)` holes where the caller's forms go.

ref-defmacro.pp

```
# `defmacro` receives its arguments as unevaluated forms (sexpr data) and
# returns a new form, expanded before the evaluator sees it. The body of
# `quasiquote { ... }` is a template in ordinary brace syntax.
defmacro unless(test, body) {
  quasiquote { if unquote(test) { nil } else { unquote(body) } }
}

print(unless(false, "ran"))
print(unless(true, "ran"))
```

```
$ pp ref-defmacro.pp
```

```
"ran"
nil
```

Because a macro's real domain is the AST, the s-expression notation remains a first-class way to write one: a `.ppl` file is the same language in AST-native form — `pp` reads it with the s-expression reader, and the template `` `(if ,test nil ,body) `` there builds the identical tree the brace template above does.

Data operations work on either origin: where quasiquote gets awkward, assemble the form directly with `list`/`cons` — `list(quote { + }, x, x)` is a complete macro body. `gensym` supplies fresh names for any binding a macro introduces; pp macros are unhygienic, so the discipline is manual.

11.14 Modules

A module is a block whose definitions become a value. `import` merges that value's exports into the current scope; a module's own body is a fresh scope.

ref-module.pp

```
# A module is a block whose definitions become a value; `import` merges that
# value's exports into the current scope. The module's body is a fresh scope.
let (geometry = module {
  def double(x) { x * 2 }
  let tau = 6
}) {
  import(geometry)
  print(double(tau))
}
```

```
$ pp ref-module.pp
```

```
12
```

`load("path.pp")` evaluates a file in the current scope. `load-module("path.pp")` evaluates it in a fresh scope and returns its definitions as a module value. `island(uri, pin)` imports a content-addressed module tree; an unpinned island is an error. The CLI and workflow details are in the Modules and islands chapter.

11.15 Desired state

`reconcile { key -> value, ... }` is identity-preserving sugar for the final desired-state map consumed by `--reconcile`; it does not perform reconciliation inside the language.

ref-reconcile.pp

```
# `reconcile` names the desired final map; the CLI reconciler consumes it.
print(reconcile { "hello.txt" -> "hello" })
```

```
$ pp ref-reconcile.pp
```

```
{"hello.txt" "hello"}
```

11.16 Lists

The core list operations are builtins and need no library: `list`, `cons`, `car`, `cdr`, and `map`.

ref-list-ops.pp

```
# List builtins need no library: list, cons, car, cdr, and map are primitive.
# `map` is deliberately lazy in the elements, serving as the parallel
# fan-out point the scheduler batches on.
print(list(1, 2, 3))
print(cons(0, list(1, 2)))
print(car(list(1, 2, 3)))
print(cdr(list(1, 2, 3)))

# map takes the function first — map(f, list) — and is lazy in the elements,
# so it is the point the scheduler batches its parallel fan-out on.
print(map(fn(x) { x * x }, list(1, 2, 3, 4)))
```

```
$ pp ref-list-ops.pp
```

```
(1 2 3)
(0 1 2)
1
(2 3)
(1 4 9 16)
```

`stdlib/list.pp` adds the higher-order functions — `foldl`, `foldr`, `filter`, `range`, `take`, `drop`, `reverse`, `length`, `nth`, `append`, `member?`, and `each`. A program loads it with `load("stdlib/list.pp")`.

ref-list-stdlib.sh

```
#!/bin/sh
# stdlib/list.pp adds the higher-order list functions. It resolves relative to
# the repo root, so the script runs pp from there.
export HOME=$(mktemp -d)
cd "$(dirname "$(dirname "$PP")")"

"$PP" /dev/stdin <<'PP'
(load "stdlib/list.pp")
(print (range 1 6))
(print (foldl + 0 (range 1 6)))
(print (filter (fn (x) (> x 2)) (list 1 2 3 4)))
(print (take 2 (list 10 20 30 40)))
(print (drop 2 (list 10 20 30 40)))
(print (reverse (list 1 2 3)))
(print (length (list 1 2 3 4)))
(print (nth 1 (list 10 20 30)))
(print (append (list 1 2) (list 3 4)))
(print (member? 2 (list 1 2 3)))
PP

rm -rf "$HOME"
```

```
pp: error: Illegal seek
```

11.17 List and vector literals

The brace surface distinguishes lists from vectors at the literal level. A bare `[...]` produces a list, while `vec[...]` produces a vector. Both support spread to splice an existing collection:

Form	Meaning
`[a, b, c]`	List literal `(list a b c)`
`vec[a, b, c]`	Vector literal `(vector a b c)`
`[head, ...tail]`	Spread: `cons(head, tail)` — only at trailing position

ref-list-vec-literals.pp

```
# List and vector literals — brackets make a list.
print([1, 2, 3])
print(vec[1, 2, 3])
```

```
$ pp ref-list-vec-literals.pp
```

```
(1 2 3)
[1 2 3]
```

11.18 Maps

Maps are immutable. The builtins construct one (`hash-map`), look up a key (`hash-map-get`), and return new maps or their components (`map-insert`, `map-remove`, `map-keys`, `map-vals`).

ref-map-ops.pp

```
# Map builtins: construct, look up, insert, and read keys/values. Maps are
# immutable — `map-insert` and `map-remove` return new maps.
# The bracket syntax `m[:key]` lowers to `hash-map-get`.
let scores = {:alice -> 95, :bob -> 87}
print(scores[:alice])

# Map update via spread — `{ ...scores, :charlie -> 78 }` produces a new map.
print({ ...scores, :charlie -> 78 })
print(map-keys(scores))
print(map-vals(scores))
print(map-remove(scores, :alice))
```

```
$ pp ref-map-ops.pp
```

```
95
{:charlie 78, :alice 95, :bob 87}
(:alice :bob)
(95 87)
{:bob 87}
```

`stdlib/map.pp` adds `map-has?` and `map-merge` on top of those builtins; it depends on `stdlib/list.pp`, so load that first.

ref-map-stdlib.sh

```
#!/bin/sh
# stdlib/map.pp builds on the map builtins and on stdlib/list.pp, so load that
# first.
export HOME=$(mktemp -d)
cd "$(dirname "$(dirname "$PP")")"

"$PP" /dev/stdin <<'PP'
(load "stdlib/list.pp")
(load "stdlib/map.pp")
(def m (hash-map :a 1 :b 2))
(print (map-has? m :a))
(print (map-has? m :z))
(print (map-merge m (hash-map :b 9 :c 3)))
PP

rm -rf "$HOME"
```

```
pp: error: Illegal seek
```

11.19 Index access

Bracket indexing works on maps and vectors, lowering to the appropriate builtin. A numeric key calls `vector-get`; any other key calls `hash-map-get`.

Form	Meaning
`m[:key]`	`hash-map-get(m, :key)` — map lookup
`v[0]`	`vector-get(v, 0)` — vector index

ref-index-access.pp

```
# Bracket indexing on maps and vectors.
let scores = {:alice -> 95, :bob -> 87}
print(scores[:alice])

let vec = vec[10, 20, 30]
print(vec[1])
```

```
$ pp ref-index-access.pp
```

```
95
[1]
```

11.20 Map update

The update syntax produces a new map with one or more keys inserted or replaced, without mutating the original.

Form	Meaning
`{ ...m, :k1 -> v1, :k2 -> v2 }`	Spread update: `map-insert(map-merge(m, new-map), k2, v2)` — rightmost wins

ref-map-update.pp

```
# Map update via spread — { ...base, key -> value } produces a new map
# with the given keys inserted or replaced.
let base = {:name -> "app", :port -> 8080}
let updated = { ...base, :port -> 9090, :debug -> true }
print(updated[:port])
print(updated[:debug])
```

```
$ pp ref-map-update.pp
```

```
9090
true
```

11.21 Strings

The string builtins cover appending, length, splitting, substrings, index-of, trimming, and conversion to and from numbers.

ref-string-ops.pp

```
# String builtins: append, length, split, substring, index-of, trim, and
# number conversion. f-strings (`f"..."`) provide interpolation.
print(string-append("foo", "bar"))
print(string-length("hello"))
print(string-split("a,b,c", ","))
print(string-sub("hello", 1, 3))
print(string-index("hello", "ll"))
print(string-trim("  hi  "))
print(number->string(42))
print(string->number("3.14"))
```

```
$ pp ref-string-ops.pp
```

```
"foobar"
5
("a" "b" "c")
"ell"
2
"hi"
"42"
3.14
```

`stdlib/string.pp` adds `string-join`, `starts-with?`, `ends-with?`, and `lines`.

`ref-string-stdlib.sh`

```
#!/bin/sh
# stdlib/string.pp adds string helpers over the string builtins.
export HOME=$(mktemp -d)
cd "$(dirname "$(dirname "$PP")")"

"$PP" /dev/stdin <<'PP'
(load "stdlib/string.pp")
(print (string-join ", " (list "a" "b" "c")))
(print (starts-with? "hello" "he"))
(print (ends-with? "hello" "lo"))
(print (lines "a
b
c"))
PP

rm -rf "$HOME"
```

```
pp: error: Illegal seek
```

11.21.1 Interpolation (f-strings)

Interpolation requires the `f` prefix glued to the opening quote. `{expr}` holes take arbitrary expressions and lower through the generic `->string` conversion; the rest of the string is literal text. Ordinary `"..."` strings never interpolate, so `"{x}"` is three literal characters — JSON fragments, shell snippets, and generated code are safe by default.

Form	Meaning
`f"a{e}b"`	`string-append("a", ->string(e), "b")`
`f"{e}"`	`->string(e)` — coerces any value to its string form
`f"{{"` / `f"}}"`	A literal `{` / `}` — doubling escapes the brace
`"{x}"`	Three literal characters — an ordinary string never interpolates

```
let name = "world"
let n = 3
print(f"Hello, {name}! Built {n} targets.")
```

`->string` renders a string as itself (no quotes) and every other value in its display form. f-strings desugar to `string-append`/`->string` with no dedicated AST node, so they round-trip through `pp fmt` with the hash preserved.

ref-fstrings.pp

```
# f-strings interpolate with the `f` prefix: `f"Hello, {name}!"`.
# Ordinary `"..."` strings never interpolate — `"{x}"` is three literal chars.
# `->string` coerces any value; a string renders as itself (no quotes).
let name = "world"
let count = 3
print(f"Hello, {name}! Built {count} targets.")
print(f"formula: {2 + 2}")
print(f"just-text")
```

```
$ pp ref-fstrings.pp
```

```
"Hello, world! Built 3 targets."
"formula: 4"
"just-text"
```

11.22 Type predicates

Each predicate forces its argument and tests its shape. The full set is `int?`, `float?`, `string?`, `bool?`, `keyword?`, `symbol?`, `pair?`, `vector?`, `map?`, `set?`, and `fn?`.

ref-predicates.pp

```
# Type predicates force their argument and test its shape.
print(int?(3))
print(string?("x"))
print(keyword?(:k))
print(pair?(list(1)))
print(map?({:a -> 1}))
print(fn?(car))
print(vector?(vec[1]))
```

```
$ pp ref-predicates.pp
```

```
true
true
true
true
true
true
true
```

11.23 Pattern matching

`match` is the one pattern-dispatch form (there is no `cond` and there are no function clauses). It tries each arm's pattern against the scrutinee in order; the first that matches wins, and a scrutinee that matches no arm is a runtime error. Patterns are literals, variables (which bind), the wildcard _, list patterns with spread (`[a, ...rest]`), and tagged patterns (`[:ok, v]`).

Form	Meaning
`match e { p => r }`	Bind `p`'s variables and evaluate `r` on the first matching arm
`p if cond => r`	**Guard:** the arm fires only when `p` matches AND `cond` (evaluated under `p`'s bindings) is truthy, else control falls to the next arm
`[a, b, ...rest] => r`	List pattern with spread — `a`, `b`, and the remainder `rest`
`[:ok, v] => r`	Tagged pattern — a two-element list headed by a keyword

Form	Meaning
`_ => r`	Wildcard — matches anything without binding

```
def classify(n) {
  match n {
    x if x < 0 => "negative"
    0          => "zero"
    x if x > 100 => "large"
    _          => "small"
  }
}
```

Guards subsume the multi-way conditional: a `match` on the scrutinized value with guarded arms replaces what a `cond` chain would have spelled. The engine agrees on every pattern kind, and the lowering uses unshadowable internal primitives — redefining `car` or `=` cannot change match semantics. The s-expression surface reads and writes the same form (`(match e (p body) (p if guard body) ...)`), so match files round-trip through `pp fmt`.

ref-match-guards.pp

```
# `match` is the one pattern-dispatch form. Guards (`p if cond => body`)
# let an arm fire only when the pattern matches AND the guard is truthy.
def classify(n) {
  match n {
    x if x < 0   => "negative"
    0            => "zero"
    x if x > 100 => "large"
    _            => "small"
  }
}

print(classify(-5))
print(classify(0))
print(classify(50))
print(classify(200))
```

```
$ pp ref-match-guards.pp
```

```
"negative"
"zero"
"small"
"large"
```

11.24 Spread

The `...` prefix is one concept in three places: list/vector construction, map update and merge (the Maps section above), and call arguments. In a list or vector literal it splices a collection in; in a call it splices arguments into the call, so a spread may appear anywhere in the argument list.

Form	Meaning
`[a, ...rest]`	`cons(a, rest)`
`vec[a, ...rest]`	`cons(a, rest)` in vector context
`f(a, ...rest, b)`	Call spread: `apply(f, list(a), rest, list(b))` — a spread may appear anywhere in the arguments

```
let flags = ["-O2", "-Wall"]
run!("cc", ...flags, "-c", src, "-o", obj)
```

A spread whose target is a compound expression uses the spaced form (`... expr`), matching the list-literal spelling.

ref-call-spread.pp

```
# The `...` prefix splices a collection into a function call:
# `f(a, ...rest, b)` lowers to `apply(f, list(a), rest, list(b))`.
# A compound spread target uses the spaced `... expr` form.
# Plain arguments before and after the spread are grouped into `list(...)`
# segments; each spread is its own segment. Here `...parts` splices the list
# into the call so `print` receives five values instead of three.
let parts = ["c", "d"]
print("a", "b", ...parts, "e")
```

```
$ pp ref-call-spread.pp
```

```
"a""b""c""d""e"
```

11.25 Observation sigils

The $ sigil introduces world observations — reads that cross the boundary between pure computation and the outside world. Every observation records the corresponding trace cell, making it visible to the cache-validity system.

Form	Lowering	Trace cell
`$file("path")`	`slurp("path")`	`file:`
`$env("VAR")`	`env-get("VAR")`	`env:`
`$env("VAR", "default")`	`if nil?(env-get("VAR")) "default" env-get("VAR")`	`env:`
`$glob("pattern")`	`perform tree-observe("pattern")`	`tree:`
`$probe("name")`	`probe("name")`	`probe:`
`$secret("path")`	`slurp("path")`	`sealed:`

ref-sigils.pp

```
# Observation sigils read the world and record trace cells.
# Each is a $ prefixed form that lowers to the corresponding builtin.
#
# $file("path")      -> slurp, records file: cell
# $env("VAR")        -> env-get, records env: cell
# $glob("pattern")   -> perform tree-observe, records tree: cell
# $probe("name")     -> probe, records probe: cell
# $config(:key)       -> config, records config: cell
# $secret("path")    -> slurp under secret grant, records sealed: cell
#
# (These require the appropriate --grant at the command line;
#  this example only demonstrates the syntax, not actual execution.)
print("sigils: $file, $env, $config, $probe, $secret")
```

```
$ pp ref-sigils.pp
```

```
"sigils: $file, $env, $config, $probe, $secret"
```

11.26 Error propagation: try

The `try` block introduces a region where bindings unwrap `[:ok, v]` / `[:err, e]` pairs automatically. Each `name <- expr` extracts the value on success or propagates the error on failure. A plain `let name = expr` inside `try` is an ordinary sequential binding that does not unwrap.

The block's last expression is the overall value (reachable only when every binding succeeded).

Form	Meaning
`try { a <- f(); body }`	Bind a to the unwrapped value; propagate on `:err`
`try { body }`	Plain body — no unwrapping, evaluates as usual

ref-try.pp

```
# try blocks unwrap [:ok, v] / [:err, e] pairs automatically.
# Each `name <- expr` extracts the value on success or propagates
# the error on failure. The block's last expression is the overall value.
let ok-result = [:ok, 42]
let err-result = [:err, "oops"]

let (good = try { a <- ok-result; a + 1 }) { print(good) }
let (bad = try { a <- err-result; a + 1 }) { print(bad) }
```

```
$ pp ref-try.pp
```

```
43
(:err "oops")
```

11.27 Collecting results

`collect` is a plain function used in pipelines: it partitions a list of `[:ok, v]` / `[:err, e]` results, returning `[:ok, values]` if every element succeeded or `[:err, errors]` if any failed. It is the validation counterpart to `try` — where `try` short-circuits at the first error, `collect` runs everything and accumulates. There is no `collect { }` block form.

Form	Meaning
`srcs |> map(f) |> collect`	`[:ok, [v…]]` if all ok, else `[:err, [e…]]`

ref-collect.pp

```
# collect partitions a list of [:ok, v] / [:err, e] results.
# It returns [:ok, values] if all succeeded or [:err, errors] if any failed.
let results = list([:ok, 1], [:ok, 2], [:err, "x"])
print(results |> collect)

let all-ok = list([:ok, 10], [:ok, 20])
print(all-ok |> collect)
```

```
$ pp ref-collect.pp
```

```
(:err ("x"))
(:ok (10 20))
```

12 Command-line reference

This appendix lists every flag pp accepts, grouped by what you use it for. It comes from the argument parser in src/app/cli.ml. Where the built-in pp --help and this table disagree, the source wins. A few flags marked internal are dispatch machinery that pp invokes on itself. They are here for completeness; you should not need to type them by hand.

Flags compose the way you would expect: --grant, --schedule, and --watch all layer onto whichever run mode you pick. Anything after a bare -- becomes the program's own argument vector, which you read with argv().

12.1 Running programs

Invocation	Meaning
pp	Start the REPL (tree-walker).
pp <file.pp>	Read and run a source file; the program's value is its last top-level form.
pp run <file>	Same as pp <file> — run is an explicit subcommand spelling.
pp -e '<expr>'	Evaluate one expression string and print each top-level value.
pp --once <file.pp>	Run once and exit. A no-op: this is already the default; the flag is for symmetry with --watch.
pp -- <args…>	Everything after -- becomes the program's argv, read via the argv() builtin.
pp --version, pp -v	Print the version and exit.
pp --help, pp -h	Print the usage summary and exit.

12.2 Execution modes

pp has a single tree-walking interpreter over the content-addressed store. Tail calls run in constant stack, and node results persist across runs.

Flag	Meaning
--once <file.pp>	Run once and exit (the default).
--watch <file.pp>	Run, then re-run when observed cells change.
--reconcile <root> <file.pp>	Treat the program's final value as a desired file tree and converge <root> to it.

12.3 Capabilities and grants

Side effects that touch the world require a capability, and capabilities enter a run only here, on the command line. --grant is repeatable; each grant is a colon-separated spec. Paths are canonicalized at the grant, so downstream authority checks and cell identities see one spelling.

Grant spec	Authority granted
fs:<path>:ro	Read access to a filesystem path (and everything under it).
fs:<path>:rw	Read and write access to a path.
fs:<path>:wo	Write-only access to a path.
net:<host>	Network access to a host, any port.
net:<host>:<port>	Network access to a host on one port.
secret:<path>	Read a path as a sealed value (bytes never enter the store; unseal is the one way out).

Grant spec	Authority granted
`process`	Spawn, signal, and reap child processes.

Usage: `pp --grant fs:/tmp/build:rw --grant process <file.pp>`. Any of the world-touching primitives (`slurp`, the `fs`/`proc` domains, network effects) raises a capability error if the matching grant is absent.

12.4 The store and auditing

Every node's result is content-addressed and cached in `~/.pp/store`. These flags inspect, bypass, audit, and reclaim that store.

Flag	Meaning
`why <file.pp>` (`--why`)	Run the file and explain each node's cache hit or miss. Capability-filtered: you only see nodes you had authority to observe.
`--no-cache <file.pp>`	Skip cache reads and recompute every node; results are still written to the store.
`--check <file.pp>`	Determinism audit: run each node twice and flag any whose result differs (a volatile node). With a non-serial `--schedule`, also re-runs the whole program serially and compares the desired-state hash. Exits 1 if anything is flagged.
`graph`	Print the cell⟶node dependency graph reconstructed from stored traces. Needs no file.
`gc`	Explicit store garbage collection (never automatic). Walks the last N wanted graphs through durable child/result/blob edges, then sweeps unreachable data.
`gc --gc-keep-epochs <N>`	Keep the last N epochs (a small built-in default; N > 0).
`gc --gc-grace-seconds <S>`	Spare anything younger than S seconds from the sweep (S ≥ 0).

12.5 Reconcile, watch, and supervise

These turn a program's value into managed state in the world. `--reconcile` materializes a `{relpath → content}` map as a file tree; `--supervise` drives a `{name → spec}` map of long-running processes. Both auto-load their domain policy from the stdlib and both are effectful, so both need grants.

Flag	Meaning
`--reconcile <root> <file.pp>`	Materialize the program's map value as a file tree under `<root>`, creating/updating/deleting by content hash. Auto-loads `stdlib/domain-fs.pp`. Needs `fs:<root>:wo` (or `:rw`).
`--supervise <file.pp>`	Converge the program's process map: start/restart/stop services to match it. Auto-loads `stdlib/domain-proc.pp`. Needs `process`. Pair with `--watch` to keep services alive.
`--watch <file.pp>`	Run, then poll the observed cells and re-evaluate whenever one changes. Combines with `--reconcile`/`--supervise` to re-converge on drift.
`--stabilize`	With `--watch`: propagate changes by dirtying only the affected nodes (push stabilization) rather than re-running cold.
`--watch-interval <s>`	Poll interval for `--watch`, in seconds (default 1.0).

Flag	Meaning
--fenced-policy retry|abort|ask	How to treat a fenced (non-convergent) action left in an unknown state by a prior crash. Default abort.

12.6 Islands

Islands are content-addressed module pins — a git ref resolved to a hash and recorded inline in the source. These flags manage the pins.

Flag	Meaning
island-pins <file.pp>	List the file's island forms with their pin and cache status. Does not run the program.
--update <file.pp>	Re-resolve each island ref and rewrite the inline pins in place, then run. Implies --fetch-islands.
--fetch-islands	Allow a git fetch for island pins not already cached (off by default — an offline run only uses what the store already holds).

12.7 Scheduling and distribution

By default, pp computes node misses serially, in-process. --schedule changes where and how they run, up to placing them on other cluster members. The M5 cluster flags establish trust (cluster-init, --mint-token) and move desired state between machines by hash (--publish-object, --desired-object).

Flag	Meaning
--schedule serial	Compute node misses one at a time, in-process (the default).
--schedule parallel:<N>	Fan out independent misses across up to N workers.
--schedule race:<N>	Race up to N redundant computations, taking the first to finish.
--schedule remote:<member>	Place misses on a named cluster member.
cluster-init	Mint ~/.pp/cluster/{secret,id} — the cluster's trust anchor.
--mint-token <out> <ttl-secs>	Mint a signed cluster token into <out>, carrying whatever --grant specs accompany it, valid for <ttl-secs>.
--member-name <n> <file.pp>	Host-qualified distribution: treat the desired state as a {host → {domain → desired}} map and converge only host <n>'s slice. Combine with --reconcile/--supervise.
--publish-object <shared-root> <file.pp>	Run the program, store its fully-forced value and canonical tree blobs in a shared local-dir store by hash, and print the hash.
--desired-object <hash> <shared-root>	Pull a published value by <hash> from <shared-root> and converge it directly — never runs a program to derive it. Takes --member-name and the reconcile/supervise flags.

12.7.1 Internal transport (not for hand use)

`pp` invokes these on itself to move artifacts and dispatch remote work. They are low-level and explicit by design; a normal run never types them.

<table>
<tr><th>Flag</th><th>Meaning</th></tr>
<tr><td><code>--transport-push object|blob|trace <id> <root></code></td><td>Copy one artifact into a shared local-dir store.</td></tr>
<tr><td><code>--transport-pull object|blob|trace <id> <root></code></td><td>Copy one artifact out of a shared local-dir store (re-hash-verified on ingest).</td></tr>
<tr><td><code>--serve-hit <key> <token-file> <shared-root> <reply-file></code></td><td>Capability-gated cache lookup served to a dispatcher.</td></tr>
<tr><td><code>--recv-hit <reply-file> <shared-root></code></td><td>Ingest a <code>serve-hit</code> reply.</td></tr>
<tr><td><code>--remote-node <token> <pins> <root> <keys> <reply></code></td><td>The cluster-member side of remote placement; authority comes from the verified token, not <code>--grant</code>.</td></tr>
</table>

12.8 Observation pinning

These capture and replay the exact set of world observations a run made — the seam that lets one machine pin another's reads (and the basis of remote placement's pin wire).

Flag	Meaning
`--pin-file <path> <file.pp>`	Preseed the run's cell observations and probe values from a file of `(pin …)` / `(pin-probe …)` lines before the program runs, so it never observes its own disk for a pinned cell.
`--dump-pins <path> <file.pp>`	After the run, write every observed cell and probe value to `<path>` as `(pin …)` / `(pin-probe …)` lines. A probe value that is not plain data (code, a handle, a sealed secret) is skipped with a warning.

The pin-file's `(pin …)` / `(pin-probe …)` lines are their own small wire format (`src/runtime/remote.ml`'s `parse_pin_line`), not pp source — they are not read by either pp reader, so they keep their fixed parenthesized shape regardless of the surface a program is written in.

13 Builtins and the standard library

This appendix indexes the two layers of pp's vocabulary that are not special forms: the builtins compiled into the binary (`src/runtime/primitives.ml`) and the standard library written in pp itself (`stdlib/*.pp`). Special forms — `if`, `let`, `def`, `fn`, `delay`, `force`, `node`, `perform`, `with-handler`, `quote`, and the rest — belong to the language reference, not here.

Signatures use pp calling syntax: `name(arg, …)`. A trailing … marks a variadic position; `[arg]` marks an optional one. Unless noted, a builtin forces the arguments it inspects; `cons`, `list`, `hash-map`, and `map` are deliberately lazy in the values they carry.

13.1 Dune adapter

`load("stdlib/dune.pp")` provides `dune-build(adapter, spec)`. `:working-tree` observes a development tree and returns selected Dune output as a canonical artifact tree. `:closed-source` executes the equivalent immutable source/tool request through `run-closed!`. The library also exposes `dune-`

`closed-request(spec)` so release requests can be inspected, hashed, and transported without execution. Dune target and output policy lives entirely in this library.

13.2 Core builtins

These are always in scope — no `load` needed.

13.2.1 Arithmetic and comparison

Signature	Description
`+(n, …)`	Variadic sum; identity 0. Mixing ints and floats promotes to float.
`-(a, b)`	Subtraction of exactly two numbers.
`*(n, …)`	Variadic product; identity 1.
`/(a, b)`	Division of two numbers; the divisor must be non-zero.
`mod(a, b)`	Integer remainder; the divisor must be non-zero.
`=(a, …)`	Structural equality; true iff all arguments are equal.
`<(a, …)`, `>(a, …)`	Variadic chained ordering: true iff each adjacent pair is strictly ordered.
`<=(a, …)`, `>=(a, …)`	As above, non-strict.

13.2.2 Lists and pairs

Signature	Description
`cons(a, b)`	Build a pair, storing both parts unforced (lazy).
`car(p)`	First element of a pair; `nil` for `nil`.
`cdr(p)`	Rest of a pair; `nil` for `nil`.
`list(x, …)`	Build a proper list from its arguments (lazy in the elements).
`apply(f, …segments)`	Apply `f` to the concatenated argument lists; used by call-spread syntax.
`map(f, lst)`	Apply `f` to each element, consing the results without forcing them — the parallel fan-out point the scheduler batches on.
`nil?(x)`	True if `x` is `nil`.

13.2.3 Vectors, maps, and sets

Signature	Description
`vector(x, …)`	Build a vector (lazy in the elements).
`vector-get(v, i)`	Element `i` of vector `v`; out-of-bounds is an error.
`hash-map(k, v, …)`	Build a map from alternating keys and values; keys are forced, values stay lazy.
`hash-map-get(m, k)`	Value bound to `k` in `m`, or `nil`.
`map-insert(m, k, v)`	A new map with `k` bound to `v` (replacing any existing binding).
`map-remove(m, k)`	A new map without key `k`.
`map-keys(m)`	The keys of `m` as a list.
`map-vals(m)`	The values of `m` as a list.
`hash-set(x, …)`	Build a set (lazy in the elements).

13.2.4 Type predicates

Each takes one argument, forces it, and returns a boolean.

Predicate	True when the value is…
`int?(x)`, `float?(x)`	an integer / a float.
`string?(x)`, `bool?(x)`	a string / a boolean.

Predicate	True when the value is…
`keyword?(x)`, `symbol?(x)`	a keyword / a symbol.
`pair?(x)`	a pair or `nil`.
`vector?(x)`, `map?(x)`, `set?(x)`	a vector / a map / a set.
`fn?(x)`	a closure or builtin.
`thunk?(x)`	an unforced thunk (does not force its argument).
`capability?(x)`	a capability.

13.2.5 Strings and numbers

Signature	Description
`string-append(s, …)`	Concatenate; non-string arguments are rendered first.
`string-length(s)`	Length of a string in bytes.
`->string(v)`	Render any value without string quotes; strings are returned unchanged.
`string-split(s, sep)`	Split `s` on the single-character `sep`, dropping empty fields.
`string-index(s, sub)`	Index of the first occurrence of `sub`, or `nil`.
`string-trim(s)`	`s` with leading and trailing whitespace removed.
`string-sub(s, start, len)`	The `len`-byte substring at `start`; out-of-bounds is an error.
`number->string(n)`	Decimal rendering of an int or float.
`string->number(s)`	Parse to an int, else a float, else `nil`.

13.2.6 I/O and control

Signature	Description
`print(x, …)`	Deep-force each argument, print them concatenated with a trailing newline, and return `nil`.
`not(x)`	Logical negation; `nil` counts as false.
`error(msg)`	Raise an error with string message `msg`.
`exit([n])`	Terminate the run with status `n` (default 0).
`slurp(path)`	Read a file to a string. Needs an `fs` read grant (or a `secret` grant, which yields a sealed value); capability-free inside a node's scratch sandbox.
`argv()`	The program arguments after `--`, as a list of strings. Recorded as an `argv:` observation.
`env-get(name)`	The environment variable `name`, or `nil`. Recorded as an `env:` observation.

13.2.7 Filesystem observation

Capability-gated presence checks, recorded as `stat:` cells so a node recomputes exactly when the path appears, disappears, or changes kind — never reading contents.

Signature	Description
`file-exists?(path)`	True if anything exists at `path`. Needs an `fs` read grant.
`dir?(path)`	True if `path` is a directory. Needs an `fs` read grant.

13.2.8 Hashing and the content store

Signature	Description
`hash-value(v)`	A canonical structural content hash of any value — order-independent for maps and sets.

Signature	Description
`hash-string(s)`	The SHA-256 hex digest of a string's raw bytes (pure; no store I/O).
`blob(s)`	Ingest bytes into the content store, returning the SHA-256 identity.
`blob-get(hash)`	The inverse of `blob`: the stored bytes for a blob identity.

13.2.9 Capabilities

Signature	Description
`current-capabilities()`	Reify the ambient capability set as of the call — an observation of the ceiling, never a mint.
`cap-compose(c, …)`	Compose several capabilities into one.
`cap-restrict(cap, scope, [:ro|:rw|:wo])`	Narrow `cap` to a scope string and, optionally, a weaker mode.
`cap-none()`	The empty capability (grants nothing).

13.2.10 Effects, domains, and metaprogramming

`stdlib/runtime.pp` provides `schedule-serial`, `schedule-parallel`, `schedule-race`, `schedule-custom`, `runtime-manifest`, reporter constructors, and policy helpers. `configure-runtime` installs a manifest at script scope. `stdlib/domain.pp` provides generic domain and probe registration helpers.

Signature	Description
`register-domain({…})`	Register a write-domain from a spec map (`:name`, `:namespace`, `:o` `cap`). Script-tier only — not inside a node body.
`register-probe(name, observe-fn, read-cap)`	Register a read-only probe: a domain with no write authority. Script-tier only.
`probe(name)`	Read a registered probe's value, pinned once per pass and recorded as a `probe:` cell.
`fenced(kind, spec)`	Register a non-convergent (fenced) action for the reconciler to drain after convergent state is applied.
`collect(results)`	Partition `[:ok, value]` and `[:err, error]` results into one aggregate result.
`unseal(v)`	Convert a sealed value to a string — the one sanctioned exit from `secret:` bytes.
`eval-pp(code)`	Parse and evaluate a code string in the current run's environment and macro table.
`apply-pp(fn, args)`	Apply `fn` to a list of already-evaluated arguments.
`force-deep(v)`	Fully force a value and everything reachable from it.
`read-string(src)`	Parse a source string to a quoted value (or a vector of them).
`gensym([prefix])`	A fresh, unwritable symbol for hygienic macro bindings.

The reader forms `quasiquote`, `unquote`, and `unquote-splicing` are also registered as builtins, and you normally reach them through the reader's quasiquote syntax; outside a quasiquote the latter two are an error. The `ppc-run` / `ppc-finish` family are self-hosting compiler scaffolding and are mostly unimplemented today.

13.3 Lists

`stdlib/list.pp` — load with `load("list.pp")`. Note that `map` is not here: it is a builtin, on purpose, so a loaded list library cannot shadow the scheduler's fan-out point.

Signature	Description
`filter(pred, lst)`	A lazy list of the elements satisfying `pred`.
`foldl(f, acc, lst)`	Left fold, strict in the accumulator.
`foldr(f, acc, lst)`	Right fold, lazy.
`range(start, end)`	The integers from `start` (inclusive) to `end` (exclusive).
`take(n, lst)`	The first `n` elements.
`drop(n, lst)`	`lst` without its first `n` elements.
`nth(n, lst)`	Zero-based element access; `nil` past the end.
`length(lst)`	Element count (strict — forces the whole list).
`each(f, lst)`	Apply `f` to each element for effect; returns `nil`.
`append(a, b)`	Concatenate two lists (lazy in `b`).
`reverse(lst)`	Strict reversal.
`member?(x, lst)`	Structural membership test.

13.4 Maps

`stdlib/map.pp` — load with `load("map.pp")`; depends on `list.pp`.

Signature	Description
`map-has?(m, k)`	Whether `k` is a key of `m`.
`map-merge(a, b)`	`a` with every binding of `b` inserted; `b` wins on collision.

13.5 Strings

`stdlib/string.pp` — load with `load("string.pp")`; no dependencies.

Signature	Description
`string-join(sep, lst)`	Join a list of strings with `sep` between elements.
`starts-with?(s, prefix)`	Whether `s` begins with `prefix`.
`ends-with?(s, suffix)`	Whether `s` ends with `suffix`.
`lines(s)`	Split `s` into lines, dropping empty fields.

13.6 Domains

The domain policies are pp libraries that `pp` auto-loads for you: `stdlib/domain-fs.pp` under `--reconcile`, `stdlib/domain-proc.pp` under `--supervise` (each after `list.pp`, `map.pp`, and `string.pp`). The trusted mechanics they call — `tree-observe`, `materialize-file`, `proc-spawn`, and so on — are OCaml primitives reached only through `perform`. You normally interact with a domain through the registration entry point; the rest are its internal policy, listed here for readers of the source.

The one entry point you call directly:

Signature	Description
`register-fs-domain(root, write-cap)`	Register the filesystem domain rooted at `root`, converging a canonical tree value.
`register-proc-domain(write-cap)`	Register the process domain, converging a `{name → spec}` map (spec: `cmd`/`args`/`env`/`cwd`).

13.6.1 Filesystem policy internals (`domain-fs.pp`)

Signature	Description
`fs-content-hash(c)`	A file descriptor's blob identity.
`fs-content-bytes(c)`	The bytes named by a file descriptor.
`fs-validate-rel-part(rel, part)`	Reject `..` in a desired path component.
`fs-validate-rel(rel)`	Validate that a desired path is relative and traversal-free.
`fs-plan-item(kind, rel, content)`	Build one create/update/delete plan item.
`fs-diff-for(root)`	Return the pure `(observed desired) → plan` diff, closed over `root`.
`fs-apply-item(root, item)`	Materialize or remove one file per plan item.
`fs-apply-for(root)`	Return the apply function that runs each plan item.

13.6.2 Process policy internals (`domain-proc.pp`)

Signature	Description
`proc-state-key(name)`	The domain-state key under which a service's record is stored.
`proc-known-names()`	The domain's own index of tracked service names.
`proc-remember!(name, pid, spec, known)`	Record a running service and add it to the known set.
`proc-forget!(name, known)`	Clear a service's record and drop it from the known set.
`proc-observe-one(name)`	Observe one service: its spec if alive, `:stopped` if tracked-but-dead, else `nil`.
`proc-observe()`	Reap zombies, then observe every tracked service.
`proc-plan-item(kind, name, spec)`	Build one start/restart/stop plan item.
`proc-spec-eq?(a, b)`	Compare two specs by canonical `hash-value` (immune to map-order differences).
`proc-diff(observed, desired)`	The pure diff: start / restart / stop by spec change.
`proc-stop-current!(name)`	Stop a service at its currently-recorded pid.
`proc-apply-item(known, item)`	Apply one plan item, returning the updated known set.
`proc-apply(plan)`	Fold every plan item, then persist the known set once.